Universidad Técnica Estatal de Quevedo
Facultad de Ciencias de la Computación
Ingeniería de Software

APLICACIONES DISTRIBUIDAS
Informe académico individual | TA-IND-01

Entrega y recuperación de mensajes en Discord

Comparación con Zulip

Estudiante
Byron Daniel Loor Mendoza
Séptimo nivel, paralelo A

Fecha de entrega
13 de noviembre de 2026

Repositorio del informe
https://github.com/Yukisan-uwo/TA-IND-01-Informe-Academico-Individual-Analisis-Critico-de-Sistemas-Distribuidos-Grupo

Resumen

Este informe analiza la entrega y el almacenamiento de mensajes de texto en Discord y los compara con Zulip, una plataforma de chat para equipos. La comparación se apoya en la documentación técnica de ambos sistemas y considera la arquitectura, los protocolos, las transparencias, la consistencia, la latencia y la recuperación ante fallos. Se realizó una prueba con un navegador en una laptop y la aplicación de Discord en un teléfono conectado por datos móviles. Durante el corte de la laptop se enviaron tres mensajes desde el teléfono. Al reconectar, los tres aparecieron en Chrome en el mismo orden, sin duplicados visibles ni recarga manual. La prueba permitió observar la recuperación del estado visible del cliente. La documentación de almacenamiento y el incidente de autenticación de Discord permiten examinar, por separado, cómo la carga, la replicación y los fallos afectan al servicio. En Zulip, el análisis distingue el historial persistente en PostgreSQL de la cola que entrega eventos. El resultado de la prueba tiene alcance local y el análisis de garantías internas depende de la operación y de la configuración documentadas.

1. Introducción

La pregunta del informe es qué compromisos entre latencia, consistencia y recuperación ante fallos se observan en la entrega y el almacenamiento de mensajes de texto en Discord, y cómo se comparan con los de Zulip. Los canales permiten que comunidades conversen en tiempo real y que quienes se conectan después consulten lo publicado. Esto requiere resolver dos problemas relacionados: conservar el historial y avisar a los clientes cuando hay actividad nueva. Discord Engineering describe una migración de un historial de billones de mensajes que pasó de 177 nodos de Cassandra a 72 de ScyllaDB, según el reporte de 2023 [1]. Zulip también ofrece chat por canales y organiza conversaciones por temas para equipos que trabajan de forma sincrónica o en horarios distintos [2].

Elegí Discord porque lo uso a diario y ya conocía su funcionamiento como usuario. Esa familiaridad me facilitó plantear una prueba con situaciones cotidianas y estudiar aspectos que antes pasaba por alto, como qué ocurre con los mensajes cuando pierdo la conexión y vuelvo a conectarme.

2. Arquitectura

La API HTTP permite solicitar el historial y crear mensajes. La arquitectura publicada por Discord ubica servicios de datos entre la API y los clústeres de ScyllaDB. Los servicios exponen operaciones gRPC y pueden reunir solicitudes simultáneas de una misma fila en una sola consulta. Discord también describe un enrutamiento por identificador de canal que ayuda a reunir solicitudes relacionadas en una instancia [1].

El Gateway entrega eventos por WebSocket. La ruta de entrega de eventos y la consulta del historial son operaciones distintas: una mantiene actualizado al cliente conectado y la otra obtiene mensajes que ya se guardaron. La API pública describe la reanudación de sesiones para aplicaciones, que usan un identificador de sesión y un número de secuencia [3]. La figura 1 digitaliza el bosquejo del Anexo A. La línea punteada indica la relación conceptual entre crear un mensaje y generar un evento para el cliente.

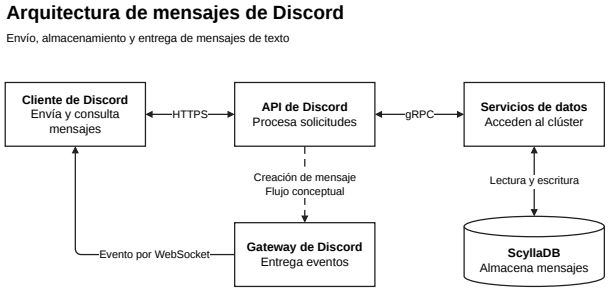


Figura 1: Flujos de solicitud, almacenamiento y entrega de eventos en Discord.

3. Conceptos distribuidos

3.1. Transparencias

Acceso. Desde el cliente se consulta un canal a través de la API. La interfaz oculta las operaciones del motor de almacenamiento.

Ubicación. El usuario trabaja con un identificador de canal. La API y el enrutamiento interno resuelven dónde atender la solicitud.

Migración. Durante el cambio de Cassandra a ScyllaDB, Discord escribió datos nuevos en ambos motores y contrastó una muestra de lecturas antes del cambio de base principal [1].

Reubicación. La documentación consultada permite analizar el traslado de datos durante la migración. El efecto de mover en caliente un servicio mientras procesa una solicitud queda sin verificar.

Replicación. El usuario ve un historial. La selección y coordinación de copias corresponde a la infraestructura, como se documenta para el clúster de autenticación [4].

Concurrencia. Solicitudes simultáneas de la misma fila pueden compartir una consulta al almacenamiento [1]. El orden observado en la prueba corresponde a un solo emisor.

Fallos. La interfaz puede mostrar la pérdida de conexión. En la prueba, la actualización del historial visible se recuperó automáticamente después de reconectar la laptop.

3.2. CAP y PACELC

Para CAP, interesa qué puede responder una operación si parte de las réplicas queda incomunicada. Discord describe lecturas por cuórum en el clúster de autenticación, con datos repartidos en tres zonas. Si una partición deja a un lado sin el cuórum requerido, una lectura en ese lado no puede completar con esa política. Durante el incidente documentado, el problema observado fue de disponibilidad de autenticación bajo mantenimiento y degradación de recursos [4]. El análisis de sistemas replicados de alta disponibilidad ayuda a identificar qué propiedades pueden debilitarse ante fallos [5].

La parte de PACELC que se aplica en operación normal pregunta cuánto cuesta coordinar réplicas frente a responder con menor latencia. En un sistema con lecturas por cuórum, esperar a varias réplicas introduce una dependencia de sus tiempos de respuesta. Las mediciones publicadas de configuraciones NoSQL muestran que el efecto de variar la consistencia depende del motor, la operación y el despliegue [6]. Discord reportó que el percentil 99 de lectura de historial pasó de un rango de 40 a 125 ms con Cassandra a 15 ms con ScyllaDB después de su migración y cambios de arquitectura [1]. Ese dato describe una mejora observada y no identifica por sí solo la política de consistencia del historial.

3.3. Modelos de comunicación

Discord sigue un modelo cliente-servidor. En HTTP, el cliente inicia la solicitud y recibe una respuesta. El Gateway usa un WebSocket persistente para entregar eventos asíncronos a aplicaciones conectadas [3]. En la ruta interna de almacenamiento, gRPC permite invocar operaciones remotas del servicio de datos [1]. La separación de eventos y solicitudes permite que distintas partes progresen a ritmos diferentes. En arquitecturas por eventos, esa separación obliga a especificar qué significa que dos componentes hayan sincronizado su estado [7].

3.4. Tolerancia a fallos

El informe de Discord sobre el 6 de noviembre de 2023 describe cómo el mantenimiento de una zona y un disco degradado en otra redujeron la capacidad del clúster de autenticación. Los reintentos añadieron carga y la interrupción afectó la lectura y el envío de mensajes [4]. Durante ese incidente, Discord observó tiempos de espera en las solicitudes de autenticación. La pérdida temporal de capacidad durante la actualización redujo el margen disponible ante el disco degradado. Tras la recuperación, el equipo amplió el clúster y revisó el proceso de actualización [4]. La replicación ayuda a tolerar la pérdida de una zona, pero la capacidad de las restantes también condiciona el servicio. Los reintentos pueden agravar la sobrecarga cuando muchas solicitudes fallan a la vez.

4. Experimento

4.1. Piloto asistido

El 12 de noviembre de 2026, Codex operó dos clientes web en una PC. Chrome recibió el mensaje de control `BYRON-PILOTO-M0`. El segundo mensaje, `BYRON-PILOTO-M1`, apareció en Chrome mientras DevTools aún mostraba `Offline`. El ajuste no aisló al receptor en esta ronda, por lo que el piloto no permite medir la recuperación tras un corte.

4.2. Prueba principal

En la primera ronda, Byron comprobó la recepción de un mensaje de control en Chrome. Durante el corte envió tres mensajes desde el celular. Una captura del navegador posterior al envío aún mostraba solo el mensaje de control de esta ronda. Al reconectar, los tres mensajes aparecieron sin recarga manual. La captura posterior muestra una entrada de cada mensaje, en orden M1, M2 y M3. La hora visible corresponde a la captura. El tiempo de recuperación quedó sin medir.

Las figuras 2 a 5 documentan el mensaje de control, el corte y el estado posterior a la reconexión. Las capturas originales están en `resultados/capturas/`.

La laptop usó Chrome y el teléfono permaneció conectado con datos móviles. Los comandos `Get-Date`, `whoami` y `Write-Output` registraron la hora, el usuario y las observaciones manuales. El procedimiento está en `resultados/EXPERIMENTO.md`. La transcripción original está en `resultados/sesion-20261112-233908.txt` e incluye una anotación posterior sobre la reconexión.

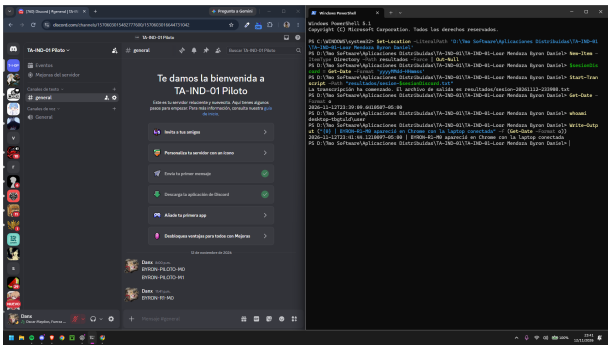


Figura 2: Mensaje de control visible en Chrome con la laptop conectada.

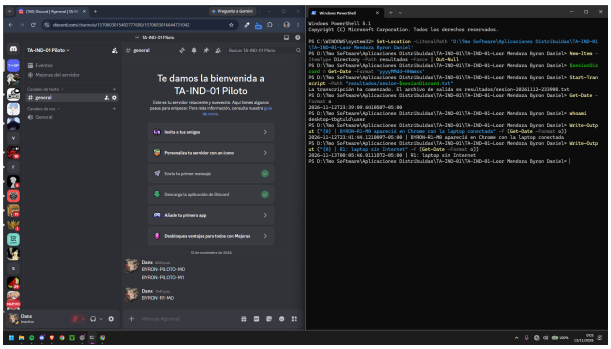


Figura 3: Inicio del corte con el indicador de red de Windows sin Internet.

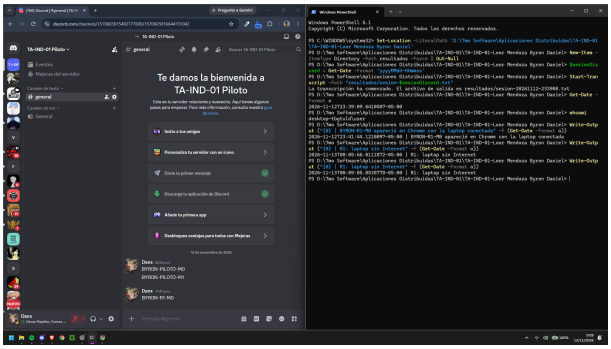


Figura 4: A las 00:09, Chrome aún muestra M0. La captura conserva el registro manual y el icono de Wi-Fi.

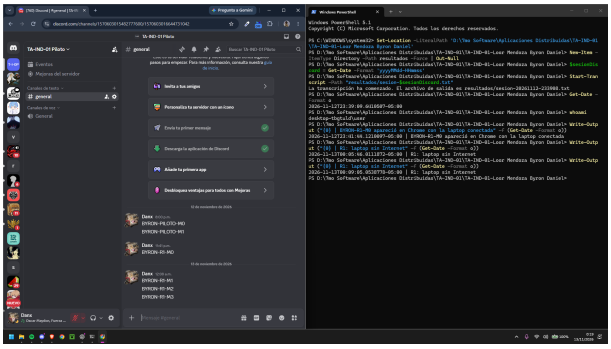


Figura 5: A las 00:19, los tres mensajes ya están visibles después de reconectar.

El resultado describe la recuperación del estado visible del cliente. La prueba interrumpió el acceso a la red de una laptop y mantuvo el teléfono conectado. Las réplicas del servicio quedaron fuera del control experimental. Por eso, la observación no permite clasificar Discord según CAP ni identificar si el cliente reanudó una sesión del Gateway o consultó el historial.

5. Ejemplos del entorno

Discord y el aviso de reconexión. Cuando se corta mi conexión, he visto el aviso «Reconectando» en la zona del micrófono de Discord. Cuando vuelve Internet, la aplicación cambia de estado. Esto ilustra una transparencia a fallos parcial: el cliente puede gestionar la reconexión y, al mismo tiempo, hacer visible la interrupción al usuario.

Zulip y el historial por temas. En la demo de Zulip, Byron envió `ZULIP-M1` al tema *Prueba TA-IND-01* del canal `#general`. Con apoyo de Codex se abrió el tema *saludos* y luego se volvió a *Prueba TA-IND-01*, donde el mensaje seguía visible. El ejemplo ilustra la transparencia de acceso: el usuario consulta el historial por canal y tema desde la interfaz, que se encarga de obtener los datos. La figura 6 muestra el resultado de la comprobación.

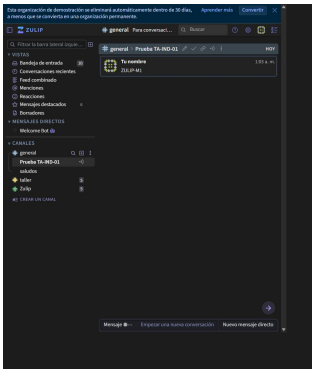


Figura 6: Vista del mensaje al volver a abrir el tema de prueba en Zulip.

6. Comparación con Zulip

La comparación considera Discord como servicio administrado y Zulip como una instalación con servidor de aplicación y PostgreSQL separados, según la configuración documentada [1], [8], [9].

Tabla 1: Comparación de la arquitectura y recuperación de mensajes en Discord y Zulip.

Discord	**Zulip**
Arquitectura	
API, servicios de datos y ScyllaDB para el historial.	Django atiende solicitudes y Tornado entrega eventos.
Comunicación	
API HTTP y eventos del Gateway por WebSocket.	Solicitudes HTTP y espera prolongada mediante *long polling*.
Transparencia de acceso	
El cliente consulta mensajes por canal mediante la API.	El usuario consulta canales y temas mediante la aplicación, que accede a la base.
Almacenamiento	
Particiones de mensajes en el clúster documentado.	PostgreSQL guarda el mensaje y su relación con los destinatarios.
Consistencia y latencia	
Lecturas por cuórum en el caso documentado de autenticación.	Escritura transaccional en el servidor primario. La configuración de replicación condiciona la recuperación.
Recuperación	
La API Gateway documenta reanudar sesiones para aplicaciones.	El cliente confirma eventos mediante su último ID. Si la cola expira, obtiene el estado inicial de nuevo.

Fuentes de la comparación [1], [3], [4], [8], [9], [10], [11]. Zulip conserva los mensajes en PostgreSQL y usa una cola por cliente para entregar las novedades. El último identificador recibido permite solicitar eventos pendientes mientras la cola existe. Si se configura una réplica de respaldo de PostgreSQL, la recuperación frente a una caída del primario implica promoverla antes de aceptar escrituras [9], [11]. Son dos recuperaciones distintas: la del cliente que perdió sus eventos y la del almacenamiento que debe volver a servir escrituras.

La demo permitió observar la organización del historial por temas en Zulip. La prueba de Discord observó la recuperación del historial visible tras un corte de conexión. La comparación de los mecanismos internos se apoya en la documentación técnica de cada plataforma.

7. Postura crítica

El diseño de Discord reduce la presión sobre el almacenamiento cuando muchos usuarios consultan los mismos mensajes. Reunir esas consultas protege a la base frente a ráfagas, aunque las particiones calientes siguen siendo un problema que el equipo debe vigilar [1]. Zulip puede ser preferible si el equipo necesita administrar el alojamiento y decidir cómo operar su PostgreSQL. Ese control añade trabajo de respaldo, supervisión y recuperación al equipo que lo aloja [2], [9].

En ambos casos conviene distinguir entre poder ver un mensaje tras reconectar y exigir una garantía para todas las operaciones ante un fallo. Una decisión que dependa de esa garantía requiere precisar la operación, la configuración de almacenamiento y las condiciones de fallo que debe soportar.

8. Conclusiones

Al reconectar la laptop, Discord mostró en Chrome los tres mensajes enviados desde el teléfono durante el corte. Aparecieron en el orden M1, M2 y M3, sin duplicados visibles ni recarga manual. En esa ronda, el cliente recuperó la vista del canal. Quedaron sin medir el tiempo de recuperación y el comportamiento ante un fallo entre réplicas.

La comparación con Zulip distingue dos tareas. La entrega de eventos mantiene actualizada la vista del usuario y el almacenamiento permite consultar el historial. Discord documenta eventos por WebSocket y servicios de datos sobre ScyllaDB. Zulip documenta una cola de eventos consultada mediante espera prolongada y un historial en PostgreSQL [1], [3], [10], [11]. La prueba de Discord permite concluir que el cliente se recuperó del corte observado. Para comparar las garantías de persistencia o la latencia de ambos sistemas harían falta pruebas con las mismas condiciones y control sobre el almacenamiento.

Declaración de uso de IA

Usé OpenAI Codex (modelo GPT-6.1 Sol) para estructurar, redactar y revisar partes del informe, revisar la consigna y cotejar referencias y DOI. Codex también digitalizó mi bosquejo en draw.io, ajustó la maquetación en LaTeX, compiló el PDF y organizó el repositorio. La bitácora registra los avances en los que redactó mensajes de commit.

Codex ejecutó el piloto asistido de Discord y preparó el mensaje de prueba de Zulip, que envié personalmente. También tomó la captura de Zulip y las del Anexo B a partir de los pasajes que marqué.

Realicé la prueba principal en mi laptop y mi celular. Comprobé en Chrome el orden de los mensajes tras reconectar y registré lo observado en PowerShell. Leí y marqué los tres artículos del Anexo B. Contrasté con Codex las referencias de Ferreira y de Bação y Guerreiro en Springer. Codex cotejó la referencia de Kokociński y colaboradores con la ficha de Elsevier y el PDF del artículo.

Referencias

[1] B. Ingram. "How Discord Stores Trillions of Messages," Discord, 6 de mar. de 2023. Consultado el 12 de nov. de 2026. [En línea]. Disponible: https://discord.com/blog/how-discord-stores-trillions-of-messages.

[2] T. Abbott. "Zulip 12.0: Organized chat for distributed teams," Zulip, 27 de abr. de 2026. Consultado el 13 de nov. de 2026. [En línea]. Disponible: https://blog.zulip.com/2026/04/27/zulip-12-0-released/.

[3] Discord. "Gateway." Consultado el 12 de nov. de 2026. [En línea]. Disponible: https://docs.discord.com/developers/events/gateway.

[4] M. Smith. "25 % or 6 to 4: The 11/6/23 Authentication Outage," Discord, 15 de nov. de 2023. Consultado el 12 de nov. de 2026. [En línea]. Disponible: https://discord.com/blog/authentication-outage.

[5] M. Kokociński, T. Kobus y P. T. Wojciechowski, "On the correctness of highly available systems in the presence of failures," *Journal of Parallel and Distributed Computing*, vol. 180, oct. de 2023, art. 104707, doi: 10.1016/j.jpdc.2023.04.008.

[6] S. Ferreira, J. Mendonça, B. Nogueira, W. Tiengo y E. Andrade, "Impacts of data consistency levels in cloud-based NoSQL for data-intensive applications," *Journal of Cloud Computing*, vol. 13, 2024, art. 158, doi: 10.1186/s13677-024-00716-7.

[7] J. L. Bação y S. Guerreiro, "Consistency challenges in event-driven microservices: a literature review on data, process, and system evolution," *Computing*, vol. 108, 2026, art. 110, doi: 10.1007/s00607-026-01701-5.

[8] Zulip. "Zulip architectural overview." Consultado el 12 de nov. de 2026. [En línea]. Disponible: https://zulip.readthedocs.io/en/stable/overview/architecture-overview.html.

[9] Zulip. "PostgreSQL database details." Consultado el 12 de nov. de 2026. [En línea]. Disponible: https://zulip.readthedocs.io/en/stable/production/postgresql.html.

[10] Zulip. "Sending messages." Consultado el 12 de nov. de 2026. [En línea]. Disponible: https://zulip.readthedocs.io/en/stable/subsystems/sending-messages.html.

[11] Zulip. "Real-time push and events." Consultado el 13 de nov. de 2026. [En línea]. Disponible: https://zulip.readthedocs.io/en/stable/subsystems/events-system.html.

A. Bosquejo inicial

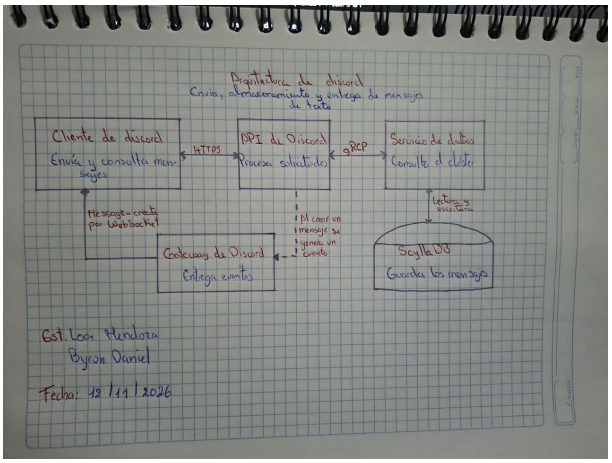
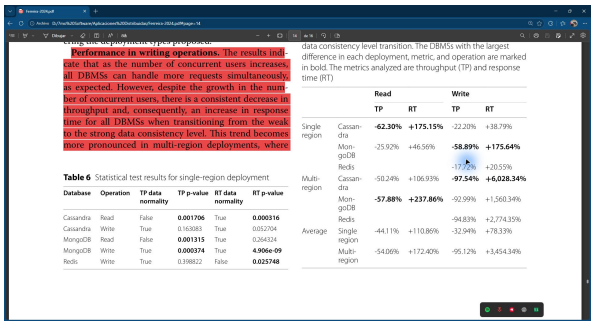


Figura 7: Bosquejo de la arquitectura de mensajes de Discord, dibujado a mano por Byron Daniel Loor Mendoza el 12 de noviembre de 2026.

B. Lectura de fuentes



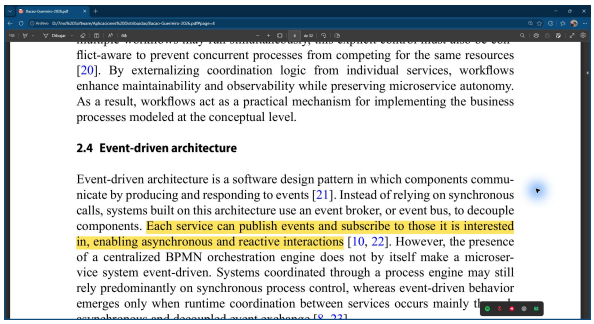
Performance in writing operations. The results indicate that as the number of concurrent users increases, all DBMSs can handle more requests simultaneously, as expected. However, despite the growth in the number of concurrent users, there is a consistent decrease in throughput and, consequently, an increase in response time for all DBMSs when transitioning from the weak to the strong data consistency level. This trend becomes more pronounced in multi-region deployments, where data consistency level transition. The DBMSs with the largest difference in each deployment, metric, and operation are marked in bold. The metrics analyzed are throughput (TP) and response time (RT)

Table 6 Statistical test results for single-region deployment

Database	Operation	TP data normality	TP p-value	RT data normality	RT p-value
Cassandra	Read	False	**0.001706**	True	**0.000316**
Cassandra	Write	True	0.163083	True	0.052704
MongoDB	Read	False	**0.001315**	True	0.264324
MongoDB	Write	True	**0.000374**	True	**4.906e-09**
Redis	Write	True	0.398822	False	**0.025748**

		Read		Write	
		TP	RT	TP	RT
Single region	Cassandra	**-62.30%**	**+175.15%**	-22.20%	+38.79%
	MongoDB	-25.92%	+46.56%	**-58.89%**	**+175.64%**
	Redis			-17.72%	+20.55%
Multi-region	Cassandra	-50.24%	+106.93%	**-97.54%**	**+6,028.34%**
	MongoDB	**-57.88%**	**+237.86%**	-92.99%	+1,560.34%
	Redis			-94.83%	+2,774.35%
Average	Single region	-44.11%	+110.86%	-32.94%	+78.33%
	Multi-region	-54.06%	+172.40%	-95.12%	+3,454.34%

Figura 8: Ferreira et al. [6], p. 14. Efecto de la consistencia fuerte sobre el rendimiento y el tiempo de respuesta en operaciones de escritura.

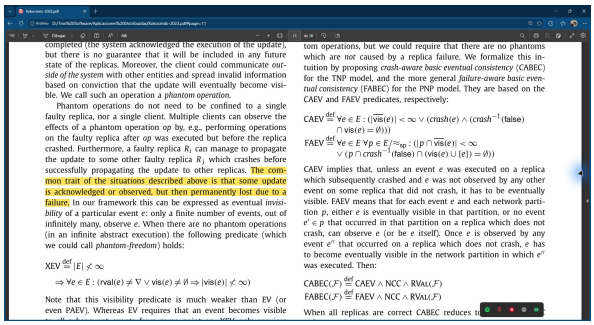


flict-aware to prevent concurrent processes from competing for the same resources [20]. By externalizing coordination logic from individual services, workflows enhance maintainability and observability while preserving microservice autonomy. As a result, workflows act as a practical mechanism for implementing the business processes modeled at the conceptual level.

2.4 Event-driven architecture

Event-driven architecture is a software design pattern in which components communicate by producing and responding to events [21]. Instead of relying on synchronous calls, systems built on this architecture use an event broker, or event bus, to decouple components. Each service can publish events and subscribe to those it is interested in, enabling asynchronous and reactive interactions [10, 22]. However, the presence of a centralized BPMN orchestration engine does not by itself make a microservice system event-driven. Systems coordinated through a process engine may still rely predominantly on synchronous process control, whereas event-driven behavior emerges only when runtime coordination between services occurs mainly th

Figura 9: Bação y Guerreiro [7], p. 4. Publicación y suscripción a eventos en la comunicación asíncrona.



completed (the system acknowledged the execution of the update), but there is no guarantee that it will be included in any future state of the replicas. Moreover, the client could communicate *outside of the system* with other entities and spread invalid information based on conviction that the update will eventually become visible. We call such an operation a *phantom operation*.

Phantom operations do not need to be confined to a single faulty replica, nor a single client. Multiple clients can observe the effects of a phantom operation *op* by, e.g., performing operations on the faulty replica after *op* was executed but before the replica crashed. Furthermore, a faulty replica R_i can manage to propagate the update to some other faulty replica R_j which crashes before successfully propagating the update to other replicas. The common trait of the situations described above is that some update is acknowledged or observed, but then permanently lost due to a failure. In our framework this can be expressed as eventual invisibility of a particular event e: only a finite number of events, out of infinitely many, observe e. When there are no phantom operations (in an infinite abstract execution) the following predicate (which we could call *phantom-freedom*) holds:

$$\mathsf{XEV} \stackrel{\text{def}}{=} |E| \not< \infty \Rightarrow \forall e \in E : (\mathsf{rval}(e) \neq \nabla \vee \mathsf{vis}(e) \neq \emptyset \Rightarrow |\mathsf{vis}(e)| \not< \infty)$$

Note that this visibility predicate is much weaker than EV (or even PAEV). Whereas EV requires that an event becomes visible

tom operations, but we could require that there are no phantoms which are *not* caused by a replica failure. We formalize this intuition by proposing *crash-aware basic eventual consistency* (CABEC) for the TNP model, and the more general *failure-aware basic eventual consistency* (FABEC) for the PNP model. They are based on the CAEV and FAEV predicates, respectively:

$$\mathsf{CAEV} \stackrel{\text{def}}{=} \forall e \in E : (|\overline{\mathsf{vis}}(e)| < \infty \vee (\mathit{crash}(e) \wedge (\mathit{crash}^{-1}(\mathsf{false}) \cap \mathsf{vis}(e) = \emptyset)))$$

$$\mathsf{FAEV} \stackrel{\text{def}}{=} \forall e \in E\ \forall p \in E / \equiv_{\mathsf{so}} : (|p \cap \overline{\mathsf{vis}}(e)| < \infty \vee (p \cap \mathit{crash}^{-1}(\mathsf{false}) \cap (\mathsf{vis}(e) \cup \{e\}) = \emptyset))$$

CAEV implies that, unless an event e was executed on a replica which subsequently crashed and e was not observed by any other event on some replica that did not crash, it has to be eventually visible. FAEV means that for each event e and each network partition p, either e is eventually visible in that partition, or no event $e' \in p$ that occurred in that partition on a replica which does not crash, can observe e (or be e itself). Once e is observed by any event e'' that occurred on a replica which does not crash, e has to become eventually visible in the network partition in which e'' was executed. Then:

$$\mathsf{CABEC}(\mathcal{F}) \stackrel{\text{def}}{=} \mathsf{CAEV} \wedge \mathsf{NCC} \wedge \mathsf{RVAL}(\mathcal{F})$$

$$\mathsf{FABEC}(\mathcal{F}) \stackrel{\text{def}}{=} \mathsf{FAEV} \wedge \mathsf{NCC} \wedge \mathsf{RVAL}(\mathcal{F})$$

When all replicas are correct CABEC reduces t

Figura 10: Kokociński et al. [5], p. 11. Pérdida de una actualización confirmada tras el fallo de una réplica.